

Tema 1 - Plataforma de desarrollo Python

Contenido Teórico

Objetivos

- **Preparar una estación de trabajo** repetible en un entorno Linux para ejecutar código Python, aislar dependencias y trabajar con notebooks.
- **Identificar qué es Python** y por qué se usa en TI empresarial.
- **Definir conceptos:** intérprete, paquetes, entornos virtuales (`venv`), IDE y notebooks.
- **Instalar Python**, `pip`, `venv`, Jupyter y Visual Studio Code en Linux.
- **Verificar el entorno** de trabajo.

Python en el contexto empresarial y TI

- **Python** es un lenguaje de programación creado a finales de los años 80 por el neerlandés [Guido van Rossum](#). El nombre se eligió en homenaje al grupo Monty Python, del que su creador es un gran fan.
- **Es de código abierto.**
- **Es multiplataforma y portable:** Linux, Windows y macOS, entre otros.
- **Es un lenguaje de propósito general:** puede usarse para automatización, integración, scripting, backend, procesamiento de datos y operaciones de sistemas.
- **Es relevante para muchos perfiles profesionales** de sistemas, cloud, seguridad, datos y desarrollo.
- **El gran crecimiento de Python se consolidó entre 2012 y 2015** con su adopción en **ciencia de datos**, gracias al ecosistema de librerías aplicables a este campo, y después se reforzó con la revolución de la **inteligencia artificial** y el **desarrollo de LLM** y tecnologías relacionadas.



Guido van Rossum, creador de Python.



Logotipo de Python

Características principales de Python

Python combina sencillez operativa con capacidad para proyectos reales.

- **Lenguaje de alto nivel:** no requiere gestionar directamente el hardware o la memoria.
- **Lenguaje interpretado:** no necesita compilarse antes de ejecutarse; se ejecuta de forma similar al scripting de shell.
- **Lenguaje legible y sencillo,** con una sintaxis clara. La indentación es obligatoria.

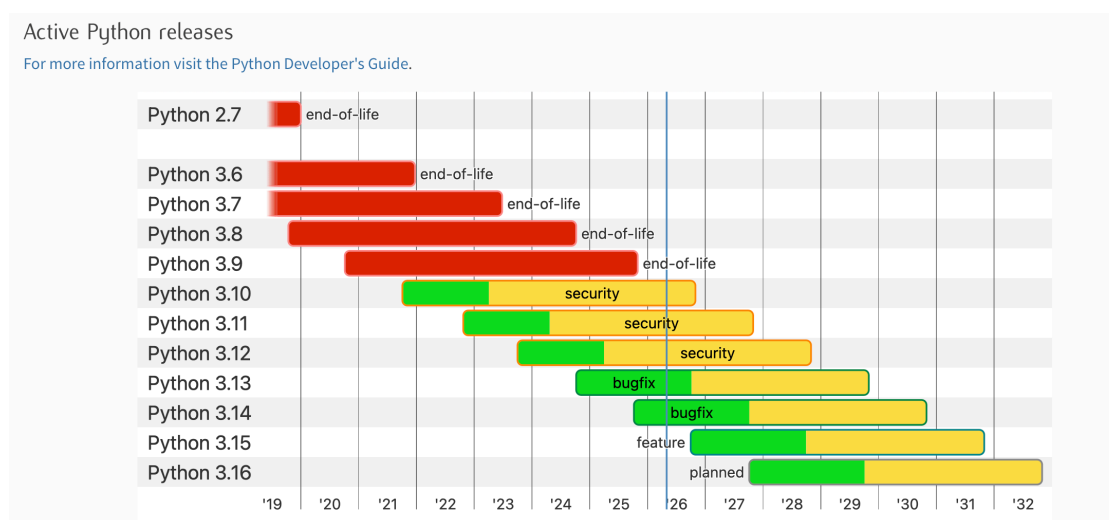
- **Tipado dinámico:** las variables no se declaran con un tipo fijo; al asignarles un valor reciben el tipo correspondiente, que puede cambiar en tiempo de ejecución.
- **Tipado fuerte:** no permite operaciones directas entre tipos incompatibles, por ejemplo, sumar una cadena y un número.
- Permite **programación orientada a objetos**, **programación procedural** y **programación funcional**.
- **Incluye una biblioteca estándar extensa** y permite usar **paquetes externos**, lo que favorece la reutilización de código.
- **Facilita la integración** con el sistema operativo, la red, archivos, APIs y herramientas externas.

Historia y evolución de Python

Año	Hito
1989	Inicio del desarrollo en CWI
1991	Primera release pública
2000	Python 2.0 Python empieza a ganar estabilidad y fuerza en desarrollo web con herramientas como Django.
2008	Python 3.0
2020	Fin de soporte de Python 2.7.18 (última versión de Python 2)
2026	Python 3.14/3.13 en soporte activo

Estado actual de las versiones de Python

Para decidir qué versión instalar o validar en un entorno corporativo conviene revisar el ciclo de vida oficial de las versiones activas.



Estado de soporte de las versiones activas de Python. Fuente: python.org/downloads.

Ventajas y cautelas con Python en la empresa

El valor no está solo en escribir código rápido, sino en mantenerlo y desplegarlo sin generar deuda técnica (Problemas de Mantenimiento a future).

Ventajas

- **Productividad:** Python permite construir soluciones con pocas líneas de código.
- **Legibilidad:** su sintaxis es clara y facilita que otros programadores entiendan el trabajo.
- **Portabilidad:** funciona en Windows, Linux y macOS.
- **Ecosistema:** existen librerías para numerosos ámbitos, como IA, finanzas, web, automatización o datos.

Cautelas

- **Dependencias no controladas:** si un programa depende de librerías externas y esas piezas cambian o desaparecen sin control, el programa puede dejar de funcionar.
- **Versiones incompatibles:** instalar una versión de un paquete puede romper otro proyecto que necesite una versión diferente.
- **Entornos globales contaminados:** instalar todo en el Python global del sistema provoca conflictos de versiones a medio plazo.

Regla de oro: trabajar siempre en entornos virtuales por proyecto. Cada proyecto debe tener su propia “burbuja” de paquetes y versiones.

Regla de soporte: documentar los comandos usados, las versiones instaladas y los paquetes requeridos para facilitar la resolución de incidencias.

Versiones: criterio de soporte

No debería instalarse “la última” versión de Python sin criterio. En un entorno profesional se usa una versión soportada y validada por el entorno corporativo.

- **Usar Python 3.x y evitar Python 2** salvo en mantenimiento legacy.
- **Comprobar ciclo de vida y soporte** antes de homologar una versión.
- En sistemas Linux empresariales puede existir un **Python instalado como parte del sistema**: no debe romperse ni sustituirse sin control.
- Para las prácticas del curso se usará la versión disponible y homologada en la VM Linux, con un **venv** separado para los laboratorios. En el entorno actual es **Python 3.13**.

Versión	Uso recomendado
Python 2.7	No iniciar proyectos nuevos
Python 3.10+	Aceptable si está soportado por SO/proyecto
Python 3.13/3.14	Adecuado si está homologado y soportado

```
$ python3 --version
$ python3 -m pip --version
```

Arquitectura de la plataforma del curso

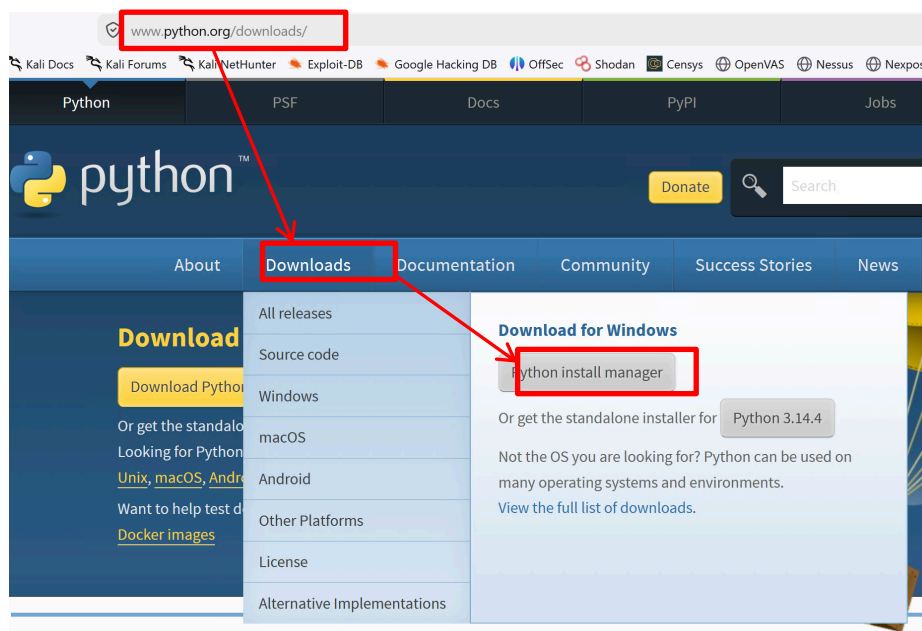
Componentes que usaremos en el laboratorio:

Componente	Función dentro del curso
Rocky Linux VM	Máquina virtual base.
Python3	Intérprete del lenguaje Python.
venv	Gestión de entornos virtuales aislados.

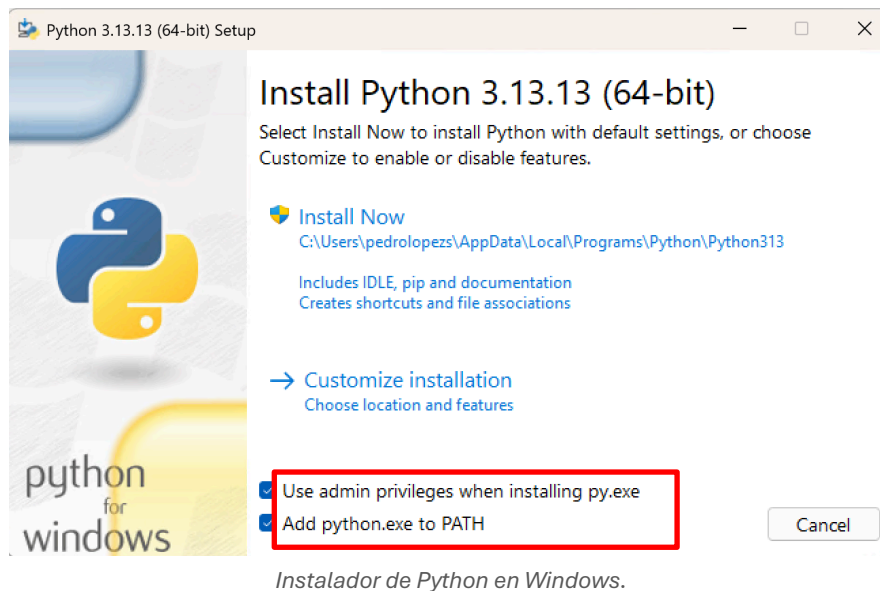
pip	Instalador de paquetes Python.
Visual Studio Code	Entorno de desarrollo para trabajar con Python.
Jupyter Notebooks	Ficheros didácticos para combinar explicación y código ejecutable.

Instalación en Windows, macOS y Linux

- Desde la web oficial de Python: <https://www.python.org>.
 - **Windows:** instalador oficial o winget; activar PATH si procede para encontrar python3 en la línea de comandos.
 - **macOS:** instalador oficial o Homebrew: `brew install python3`.
 - **Linux:** gestor de paquetes de la distribución. No sustituir Python del sistema ni módulos sin control. Usar venv.
- **Laboratorio del curso:** máquina virtual Rocky Linux entregada por el instructor. **Ya lleva preinstalado Python 3.13, las dependencias para venv y Jupyter, y el entorno de trabajo Visual Studio Code.**



Descarga de Python desde python.org.



Comandos base en Linux para empezar con Python

La máquina virtual para el curso permitirá que todos trabajemos con la misma base. No necesitas grandes conocimientos de linux. usaremos comandos básicos

Secuencia mínima inicial para preparar la VM Linux para trabajar con Python:

- Comprobar el sistema operativo y el kernel.
- Actualizar el sistema.
- Instalar herramientas base.
- Verificar python3, pip y venv.
- Clonar el repositorio Git para el curso.

```
# Comprobar el sistema operativo y actualizarlo
cat /etc/os-release
uname -r
sudo dnf update -y

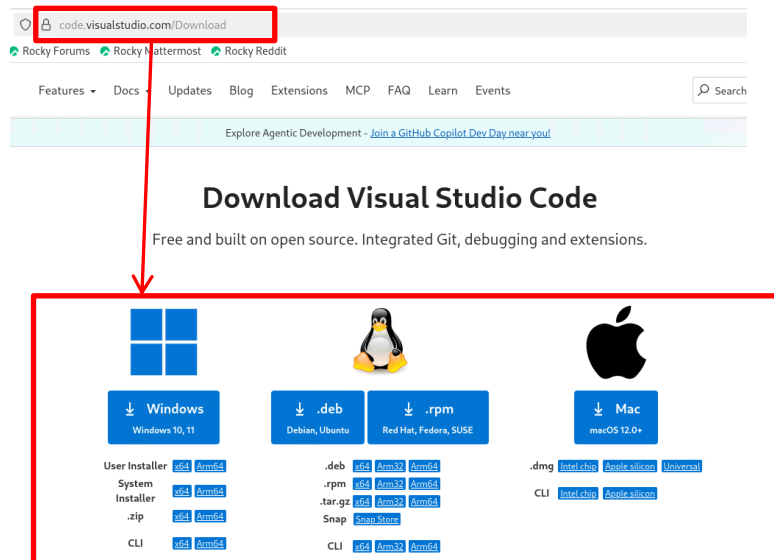
# Instalar herramientas base
sudo dnf install -y python3.13 python3.13-pip python3-virtualenv git

# Verificación: la versión Python del sistema es la 3.9,
# pero trabajaremos con la 3.13
which python          # la respuesta es /usr/bin/python
ls -l /usr/bin/python  # /usr/bin/python -> ./python3
ls -l /usr/bin/python3 # /usr/bin/python3 -> python3.9
which python3.13      # /usr/bin/python3.13
python3.13 --version
python3.13 -m pip --version

# Clonación del directorio Git para el curso
# Nos ubicamos en /home/user
cd .
# Clonamos el repositorio Git para el resto del curso
# Nota: encontrarás el repositorio en el documento de laboratorio
git clone URL_PARA_EL_CURSO Curso_Python
# A partir de ahora trabajaremos siempre en este directorio
cd Curso_Python
```

Instalación de Visual Studio Code

- **Visual Studio Code es un IDE/editor de desarrollo** pensado para trabajar con diferentes lenguajes.
- **Es gratuito.**
- Puede instalarse desde la web oficial: <https://code.visualstudio.com/>.
- Windows y macOS tienen instalador integrado.
- En Linux puede instalarse desde paquetes .rpm o .deb, o desde la línea de comandos.



Descarga de Visual Studio Code por sistema operativo.

RHEL, Fedora, and CentOS based distributions

We currently ship the stable 64-bit VS Code for RHEL, Fedora, or CentOS based distributions in a yum repository.

- 1 Install the key and yum repository by running the following script:

Bash

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc &&
echo -e "[code]\nname=Visual Studio Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nautorefresh=1\
ntype=rpm-md\ngpgcheck=1\ngpgkey=https://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/vscode.repo
```

- 2 Then update the package cache and install the package using dnf (Fedora 22 and above):

Bash

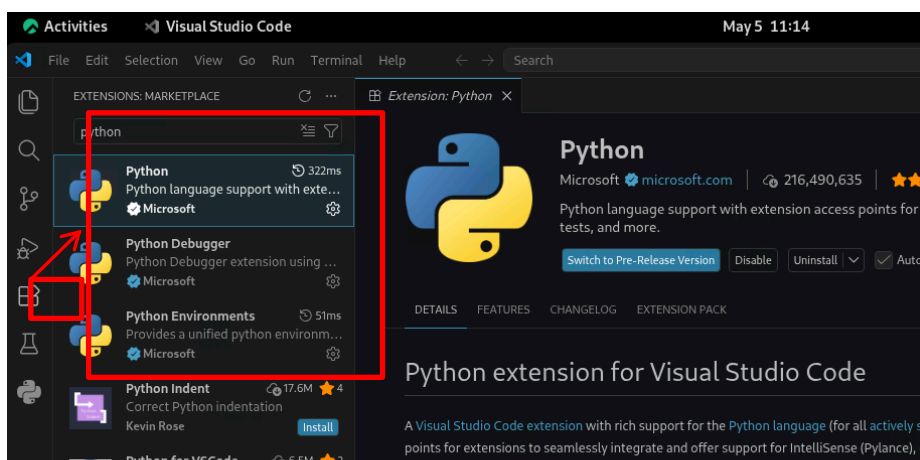
```
dnf check-update &&
sudo dnf install code # or code-insiders
```

Instalación de Visual Studio Code en distribuciones Linux basadas en RPM.

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc && echo -e \
"[code]\nname=Visual Studio \
Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\nautorefresh=1\
ntype=rpm-md\ngpgcheck=1\ngpgkey=https://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/vscode.repo
sudo tee /etc/yum.repos.d/vscode.repo > /dev/null
dnf check-update &&sudo dnf install code
```

Extensión de Python en VS Code

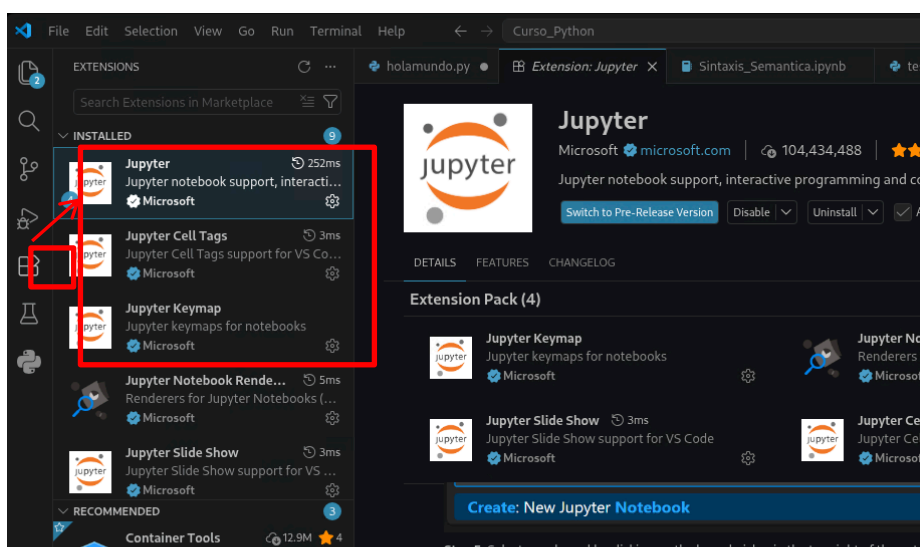
Una vez instalado VS Code, se instala la extensión de Python. Esta extensión permite resaltado de código, creación de entornos virtuales y depuración. En el entorno de laboratorio del curso, Visual Studio Code y la extensión de Python ya vienen instalados.



Extensión Python para Visual Studio Code.

Extensión de Jupyter en VS Code

Los Jupyter Notebooks son ficheros con extensión .ipynb que combinan celdas de código ejecutable con celdas de texto y marcas. Los usaremos para la parte de ejercicios guiados. Es necesario instalar la extensión Jupyter en VS Code; en el entorno de laboratorio del curso ya está instalada.



Extensión Jupyter para Visual Studio Code.

Entornos virtuales con venv

- Un venv crea un entorno aislado con su propio ejecutable Python y sus paquetes instalados desde `pip`.
- Evita instalar librerías en el Python del sistema. Debe activarse antes de instalar dependencias del proyecto.
- Permite reproducir proyectos con el fichero `requirements.txt`.
- Reduce conflictos entre versiones de paquetes.
- Facilita la portabilidad: otros entornos pueden reproducir el proyecto con las mismas versiones.
- Para lanzar Visual Studio Code con un venv, lo más sencillo es invocarlo desde el directorio del proyecto con el comando `code .`
- Para volver al entorno del sistema se usa la función `deactivate`.

```
# Verificar que estamos en el directorio para el curso
pwd # /home/user/Curso_Python

# Crear la infraestructura del venv
python3.13 -m venv .venv --prompt "Curso_Python"
ls -l .
tree . # Muestra las dependencias instaladas
python --version # Responde con Python 3.9

# Activamos el venv
# NOTA: en Windows es .venv\Scripts\activate
source .venv/bin/activate
python --version # Responde con Python 3.13

# Actualizamos pip
python -m pip install --upgrade pip

# Para ejecutar Jupyter Notebooks instalamos:
pip install notebook ipykernel

# Lanzamos Visual Studio Code con el venv
code .

# Para desactivar el venv y volver a trabajar con Python del sistema
deactivate
```


pip y control de dependencias

- `pip` es el instalador de paquetes desde índices como PyPI (Python Package Index) o repositorios corporativos locales.
- Regla: instalar dentro del `venv`, no globalmente.
- Actualizar `pip` dentro del entorno cuando sea necesario.
- Instalar paquetes con `pip` dentro del `venv`.
- Registrar dependencias con `pip freeze` o herramientas equivalentes.
- En lugar de guardar todo el `venv`, que puede ocupar mucho espacio, se entrega el fichero de requerimientos junto con los ficheros de código.
- En entornos empresariales puede usarse un mirror interno o proxy autenticado para `pip`.

```
# Activar y actualizar pip en el venv
# (Ya lo hicimos antes)
source .venv/bin/activate
python --version
python -m pip install --upgrade pip

# Instalar el paquete cowsay, listar y probar
pip install cowsay
pip list
python -c 'import cowsay; cowsay.cow("Hola!")'

# Crear un fichero de requerimientos y verlo
pip freeze > requirements.txt
cat requirements.txt

# Desinstalar un paquete
pip uninstall cowsay

# Recuperar los paquetes de requerimientos
pip install -r requirements.txt
```

Otros entornos

Además de las herramientas de trabajo del curso, existen otros entornos de desarrollo o ejecución que conviene identificar.

Herramienta	Uso principal	Descripción	URL
IDLE	Pruebas simples	Entorno de desarrollo básico y ligero que se instala por defecto con Python (Paquete <code>python3-idle</code>). Incluye un editor de texto sencillo y una shell interactiva para ejecutar comandos rápidos.	docs.python.org/3/library/idle
VS Code	Scripts/proyectos	Editor de código fuente ligero pero muy potente y extensible mediante extensiones. Es el estándar actual por su versatilidad, autocompletado inteligente y excelente integración con terminales y entornos virtuales.	code.visualstudio.com
JupyterLab	Notebooks	Entorno interactivo basado en web que permite combinar bloques de código vivo con texto enriquecido (Markdown), visualizaciones y fórmulas. Es la evolución de Jupyter Notebook y es ideal para experimentación y narrativa de datos.	jupyter.org
PyCharm	IDE profesional	Entorno de desarrollo integrado (IDE) completo, diseñado específicamente para programadores de Python. Ofrece	jetbrains.com/pycharm

		herramientas avanzadas de depuración, análisis de código y soporte para frameworks profesionales como Django o Flask.	
Anaconda/Spyder	Datos/ciencia	Distribución que incluye un gestor de paquetes propio (Conda) y Spyder, un IDE orientado a la computación científica. Proporciona un entorno robusto para análisis de datos, ingeniería y ciencia con librerías preinstaladas.	anaconda.com

Flujo de trabajo estándar del curso

Se recomienda este flujo de trabajo en cada tema:

- La base de los laboratorios estará en el directorio `/home/user/Curso_Python`, descargado desde el repositorio Git facilitado en el curso.
- Dentro crearemos y activaremos el `.venv` para Python 3.13 si no está creado aún.
- Instalaremos las dependencias necesarias.
- Usaremos una carpeta de trabajo por tema dentro del repositorio: Tema01, Tema02, Tema03, etc.
- Ejecutaremos scripts `.py` y Jupyter Notebooks `.ipynb` dentro del `venv`.

Errores frecuentes de entorno

Muchos fallos iniciales no son de Python, sino de la plataforma.

- Usar `pip` de otro Python distinto.
- Instalar paquetes fuera del `venv`.
- No activar el entorno antes de ejecutar código Python o Jupyter.
- Confundir `python`, `python3`, `pip` y `pip3`. Si no hay otro Python legacy, el comando `python` suele estar enlazado a `python3` del sistema.
- No registrar `requirements.txt` si el `venv` se ha limpiado y solo tenemos el fichero de requerimientos.
- Recomendación: al migrar un proyecto de código, no empaquetar el directorio `.venv` porque ocupa demasiado espacio. El fichero `requirements.txt` es suficiente para recrear el entorno de requerimientos con `pip`.

Conclusiones

Al finalizar este tema, debes tener una plataforma homogénea y verificable.

- VM Linux Rocky operativa.
- Python, `pip`, `venv` y Jupyter instalados.
- Base preparada para sintaxis y ejercicios en los siguientes temas.